



1



2



Giới thiệu Django Rest API

- Django REST framework (DRF) là bộ công cụ mạnh và linh hoạt để phát triển API.
- Một số ưu điểm sử dụng Django REST API.
 - Hỗ trợ giao diện duyệt API hiệu quả cho dev.
 - Hỗ trợ nhiều authentication policies, chứng thực bao gồm OAuth1a và OAuth2.
 - Serialization hỗ trợ cả các data source theo ORM và không ORM (non-ORM).
 - Tài liệu phong phú và cộng đồng lớn.

3

3



Cài đặt Django Rest API

- Cài đặt môi trường

```
pip install django
pip install djangorestframework
pip install markdown
pip install django-filter
```

- Cập nhật biến INSTALLED_APPS

```
INSTALLED_APPS = [
    ...
    'rest_framework',
]
```

4

4



Viết API đầu tiên

- Tạo `courses/serializers.py`

```
from rest_framework.serializers import ModelSerializer
from .models import Course

class CourseSerializer(ModelSerializer):
    class Meta:
        model = Course
        fields = ['id', 'subject', 'created_date', 'category']
```

5

5



Viết API đầu tiên

- Viết `courses/views.py`

```
from rest framework import viewsets, permissions
from .models import Course
from .serializers import CourseSerializer

class CourseViewSet(viewsets.ModelViewSet):
    queryset = Course.objects.filter(active=True)
    serializer_class = CourseSerializer
    permission_classes = [permissions.IsAuthenticated]
```

6

6

Viết API đầu tiên

- Viết `courses/urls.py`

```
from django.urls import path, include
from . import views
from rest framework import routers

router = routers.DefaultRouter()
router.register('courses', views.CourseViewSet)

urlpatterns = [
    path('', include(router.urls))
]
```

- Chạy server và truy cập `http://127.0.0.1:8000/`

7

7

Viết API đầu tiên

The screenshot shows a web browser window with the address bar displaying `127.0.0.1:8000`. The page title is "Django REST framework" and the user is logged in as "admin". The main content area shows the "Api Root" page, which includes a search bar, a "OPTIONS" button, and a "GET" button. Below the buttons, the response for the GET request is displayed: `HTTP 200 OK`, `Allow: GET, HEAD, OPTIONS`, `Content-Type: application/json`, and `Vary: Accept`. The JSON response is `{ "courses": "http://127.0.0.1:8000/courses/" }`.

8

8

Viết API đầu tiên

The screenshot shows the Django REST framework interface for the 'Course List' endpoint. The browser address bar shows '127.0.0.1:8000/courses/'. The page title is 'Django REST framework' with a user 'admin' logged in. The breadcrumb is 'Api Root / Course List'. The endpoint is 'GET /courses/'. The response is 'HTTP 200 OK' with 'Allow: GET, POST, HEAD, OPTIONS', 'Content-Type: application/json', and 'Vary: Accept'. The JSON response is:

```
[{"id": 1, "subject": "Các công nghệ lập trình hiện đại", "created_date": "2021-05-29T01:35:34.495737Z", "category_id": 1}]
```

At the bottom, there is a 'Raw data' tab and an 'HTML form' tab. The 'HTML form' tab is active, showing a 'Subject' input field and a 'POST' button.

9

Viết API đầu tiên

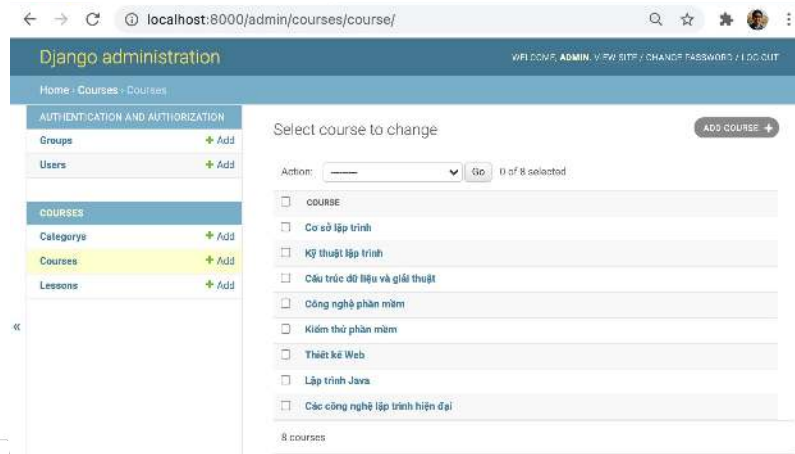
The screenshot shows the Django REST framework interface for the 'Course List' endpoint. The browser address bar shows '127.0.0.1:8000/courses/'. The page title is 'Django REST framework'. The breadcrumb is 'Api Root / Course List'. The endpoint is 'GET /courses/'. The response is 'HTTP 403 Forbidden' with 'Allow: GET, POST, HEAD, OPTIONS', 'Content-Type: application/json', and 'Vary: Accept'. The JSON response is:

```
{ "detail": "Authentication credentials were not provided." }
```

10

Viết API đầu tiên

- Phân trang cho phép chỉ định số phần tử hiển thị trên từng trang.
- Truy cập vào admin thêm vài khoá học



11

Viết API đầu tiên

- Ta phân mỗi trang hiển thị tối đa 2 khoá học.
- Trong tập tin settings.py thiết lập cấu hình sau

```
REST_FRAMEWORK = {  
    'DEFAULT_PAGINATION_CLASS':  
        'rest_framework.pagination.PageNumberPagination',  
    'PAGE_SIZE': 2  
}
```

- Truy cập <http://localhost:8000/courses/>

12

Viết API đầu tiên

Click vào đây xem các trang hiển thị khoá học

```

HTTP 200 OK
Allow: GET, POST, HEAD, OPTIONS
Content-Type: application/json
Vary: Accept

{
  "count": 4,
  "next": "http://localhost:8000/courses/?page=2",
  "previous": null,
  "results": [
    {
      "id": 1,
      "subject": "Các công nghệ lập trình hiện đại",
      "created_date": "2021-05-29T01:35:34.495737Z",
      "category_id": 1
    },
    {
      "id": 2,
      "subject": "Lập trình Java",
      "created_date": "2021-06-01T03:09:56.346259Z",
      "category_id": 1
    }
  ]
}
  
```

```

HTTP 200 OK
Allow: GET, POST, HEAD, OPTIONS
Content-Type: application/json
Vary: Accept

{
  "count": 8,
  "next": "http://localhost:8000/courses/?page=4",
  "previous": "http://localhost:8000/courses/?page=2",
  "results": [
    {
      "id": 5,
      "subject": "Công nghệ phân nh\u00e0",
      "created_date": "2021-06-01T03:11:48.287315Z",
      "category_id": 1
    },
    {
      "id": 6,
      "subject": "Câu tr\u00e0c \u0111\u1ee9 li\u1ec7u v\u00e0 gi\u00e1i th\u00f9a",
      "created_date": "2021-06-01T03:12:49.412186Z",
      "category_id": 1
    }
  ]
}
  
```

13

Request

- Request của REST framework **kế thừa từ HttpRequest chuẩn**, nhưng linh hoạt hơn trong xử lý và chứng thực request (request parsing và request authentication).
- Request giúp cho việc xử lý dữ liệu request dạng json hay một media type nào khác theo **cùng cách thức** làm việc với dữ liệu form.

14



Các thuộc tính của Request

- **request.data**: dữ liệu của request body.
- **request.query_params**: các tham số truyền từ request get.
- **request.method**: trả về phương thức (chữ in hoa) thực hiện request như GET, POST, PUT, DELETE, PATCH.
- **request.content_type**: trả về chuỗi đại diện cho media type của request body.

15

15



Các thuộc tính của Request

- **request.user**: các user đã chứng thực thì nó trả về thể hiện của
 - **django.contrib.auth.models.User**
- Ngược lại, nó là thể hiện của
 - **django.contrib.auth.models.AnonymousUser**
- **request.auth**: chứa thông tin bổ sung chứng thực, nó phụ thuộc cách thức chứng thực, nhưng nó **thường chứa thông tin token** của request.

16

16



Các thuộc tính của Request

- **request.authenticators**: các chính sách chứng thực, trong các lớp APIView hoặc @api_view thuộc tính được thiết lập thông qua thuộc tính authentication_classes hoặc giá trị thuộc tính cấu hình DEFAULT_AUTHENTICATORS.

17

17



Response

- REST framework cung cấp lớp Response trả về nội dung với nhiều dạng media type phụ thuộc client request. Lớp Response là lớp con của Django SimpleTemplateResponse.
- Cú pháp tạo response:

```
Response(data, status=None, template_name=None,  
         headers=None, content_type=None)
```

18

18



Các thuộc tính của Response

- **data**: dữ liệu đã được serialize.
- **status**: status code cho response, mặc định 200.
- **headers**: từ điển HTTP headers cho response.
- **content_type**: content type cho response.
- **template_name**: tên template sử dụng nếu HTMLRenderer được chọn.

19

19



Serializer

- **Serializer** cho phép **chuyển các dữ liệu** phức tạp như querysets hoặc thể hiện của model thành các kiểu dữ liệu python và dễ dàng **kết xuất thành dữ liệu JSON**, XML hay một content type nào khác.
- Nó cũng cung cấp khả năng **deserializer** để chuyển dữ liệu đã được **chuyển về kiểu dữ liệu phức tạp ban đầu** sau khi đã kiểm tra (validate).

20

20



Serializer

- Phương thức **validate()** dùng để kiểm tra chung cho nhiều trường trong serializer, phương thức có một đối số duy nhất là từ điển chứa giá trị các trường.
- Phương thức **validate_<field_name>** sẽ trả về giá trị kiểm tra của trường tương ứng hoặc ném ngoại lệ `serializers.ValidationError`.
- Chú ý nếu `<field_name>` được khai báo `required` là `False` thì `validate` sẽ không được thực hiện.

21

21



Serializer

- **Lớp Serializer** cung cấp các phương thức chung để xử lý output cho response.
- **Lớp ModelSerializer** giúp nhanh chóng tạo lớp Serializer với **các trường tương ứng với các trường của Model**. Khi đó
 - Tự tạo các field giống model.
 - Tự tạo validator cho serializer.
 - Bao gồm các phương thức mặc định như `save`, `update`, `create`.

22

22



Serializer

- **Lớp HyperlinkedModelSerializer** tương tự ModelSerializer, nhưng nó sử dụng siêu liên kết đại diện cho các quan hệ.

23

23



ModelSerializer

- Thuộc tính trong meta class
 - **model**: lớp model liên kết cho serialize.
 - **fields**: chỉ định các field sẽ được serialize, thiết lập là '__all__' để serializer tất cả field.
 - **read_only_fields**: chỉ định các field chỉ đọc.
 - **exclude**: chỉ định các field không serialize.
 - **extra_kwargs**: bổ sung một số thông tin ràng buộc trên fields.
 - **deep**

24

24



ModelSerializer

- Ví dụ tạo serializer cho Lesson

```
class TagSerializer(ModelSerializer):  
    class Meta:  
        model = Tag  
        fields = ['id', 'name']  
  
class LessonSerializer(ModelSerializer):  
    tags = TagSerializer(many=True)  
  
    class Meta:  
        model = Lesson  
        fields = ['id', 'subject', 'content',  
                  'created_date', 'updated_date', 'tags']
```

25

25



ModelSerializer

- Sử dụng cho 1 đối tượng QuerySet

```
lesson = Lesson.objects.get(pk=1)  
LessonSerializer(lesson).data
```

- Sử dụng cho danh sách QuerySet

```
lessons = Lesson.objects.filter(active=True)  
LessonSerializer(lessons, many=True).data
```

- Cung cấp thêm thông tin context cho serializer

```
LessonSerializer(lesson,  
                 context={'request': request}).data
```

26

26

ModelSerializer

Lesson Instance

GET /lessons/1/

```

HTTP 200 OK
Allow: GET, HEAD, OPTIONS
Content-Type: application/json
Vary: Accept

{
  "id": 1,
  "subject": "Giới thiệu",
  "content": "Giới thiệu các công nghệ lập trình hiện đại",
  "created_date": "2021-05-30T01:59:15.931582Z",
  "updated_date": "2021-05-30T07:41:17.071208Z",
  "tags": [
    {
      "id": 1,
      "name": "nhập môn"
    }
  ]
}

```

27

ModelSerializer

- Sau đây 2 serializer minh họa việc chỉnh sửa hoặc ghi đè xử lý của serializer
 - Ví dụ 1: chỉnh sửa thông tin serialize image của CourseSerializer trả về đường dẫn tuyệt đối cho client.
 - Ví dụ 2: ghi đè phương thức create của UserSerializer trong chức năng đăng ký người dùng để băm mật khẩu trước khi lưu xuống CSDL.

28



ModelSerializer

```
class CourseSerializer(HyperlinkedModelSerializer):
    image = serializers.SerializerMethodField(source='image')

    def get_image(self, obj):
        request = self.context['request']
        if obj.image.name.startswith('static/'):
            path = "/" %s % obj.image.name
        else:
            path = '/static/%s' % (obj.image)

        return request.build_absolute_uri(path)

class Meta:
    model = Course
    fields = ['id', 'subject', 'image',
              'created_date', 'category id']
```

29



ModelSerializer

```
class UserSerializer(ModelSerializer):
    class Meta:
        model = User
        fields = ['id', 'first_name', 'last_name',
                  'username', 'password', 'email', 'avatar']
        extra_kwargs = {'password': {'write_only': True}}

    def create(self, validated_data):
        data = validated_data.copy()

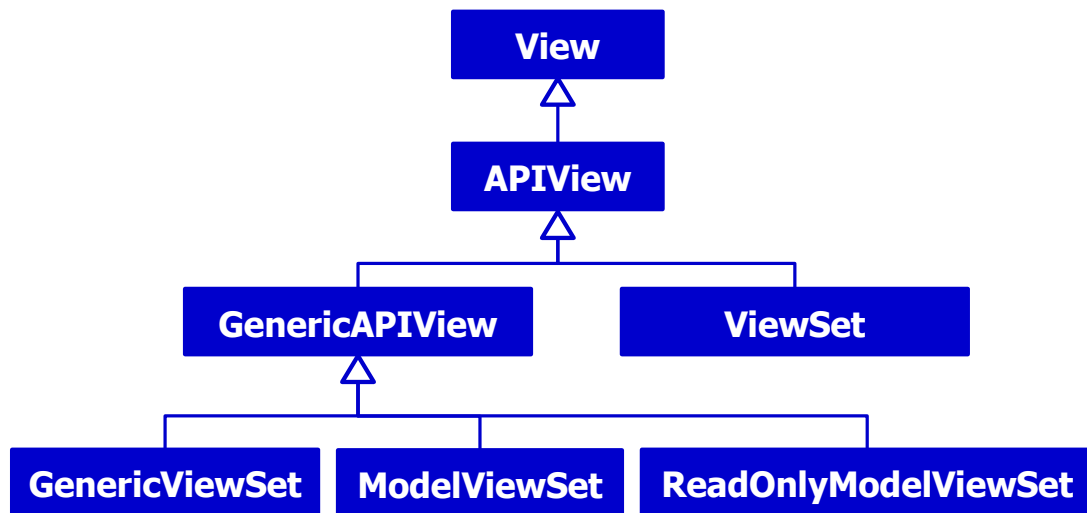
        user = User(**data)
        user.set_password(user.password)
        user.save()

        return user
```

30

30

View



31

31

APIView

- REST framework cung cấp **lớp APIView** là lớp con của lớp View trong Django. Trong đó
 - Đối tượng request là thể hiện Request của REST framework.
 - Các phương thức trả về response là thể hiện Response của REST framework.
 - Sử dụng cũng tương tự View sẽ có các phương thức request tương ứng như `get()`, `post()` và nhiều thuộc tính cho API policy.

32

32



APIView

```
from rest_framework.views import APIView
from rest_framework.permissions import IsAuthenticated

class CommentAPIView(APIView):
    def get_permissions(self):
        if self.request.method == 'GET':
            return [permissions.AllowAny()]

        return [permissions.IsAuthenticated()]

    def get(self, request, lesson_id):
        pass

    def post(self, request, lesson_id):
        pass
```

33

33



APIView

- Viết phương thức get() của CommentAPIView

```
class CommentAPIView(APIView):
    ...
    def get(self, request, lesson_id):
        comments = Comment.objects.filter(lesson_id=lesson_id)
        serializer = CommentSerializer(comments, many=True)

        return Response(serializer.data,
                        status=status.HTTP_200_OK)
    ...
```

34

34

```
class CommentAPIView(APIView):
    ...
    def post(self, request, lesson_id):
        content = request.data.get('content')
        if content is not None:
            try:
                c = Comment.objects.create(content=content,
                                           user=request.user,
                                           lesson_id=lesson_id)

            except IntegrityError:
                err_msg = "Lesson does not exist!"
            else:
                return Response(CommentSerializer(c).data,
                                status=status.HTTP_201_CREATED)
        else:
            err_msg = "Content is required!!!"

        return Response(data={'error_msg': err_msg},
                        status=status.HTTP_400_BAD_REQUEST)
```

35

- Ngoài ra ta có thể viết view bằng function

```
from rest_framework.decorators import api_view

@api_view(['GET', 'POST'])
@permission_classes([IsAuthenticated])
def comment_api_view(request):
    if request.method == 'GET':
        pass
    elif request.method == 'POST':
        pass
```

36

36



APIView

- Cập nhật `courses/urls.py`

```
urlpatterns = [  
    ...  
    path('lessons/<int:lesson_id>/comments/',  
        views.CommentAPIView.as_view()),  
]
```

- Khi đó, ta có các API sau:
 - `/lessons/{lesson_id}/comments` – GET
 - `/lessons/{lesson_id}/comments` – POST

37

37



Generic View

- Generic View giúp phát triển nhanh API view.
- `GenericAPIView` kế thừa `APIView`, bổ sung thêm một số hành vi chuẩn như `list` và `detail`.

38

38



Generic View

- **queryset**: trả về các đối tượng từ view. Queryset sẽ được thực thi một lần khi nó được sử dụng (evaluated) và nó sẽ lưu đệm (cached) cho các lần truy vấn sau. Nếu có ghi đè phương thức **get_queryset()** thì kết quả sẽ ưu tiên cao hơn.
- **serializer_class**: lớp serializer được sử dụng validate, deserialize input và serialize output. Ta có thể ghi đè phương thức **get_serializer_class()** thay thế.

39

39



Generic View

- **lookup_field**: các trường dùng lọc dữ liệu trong các thể hiện model, mặc định là pk.
- **lookup_url_kwarg**: các đối số URL argument.
- **pagination_class**: chỉ định lớp để phân trang cho kết quả list, mặc định sử dụng giá trị cấu hình của DEFAULT_PAGINATION_CLASS. Nếu `paginate_class=None` sẽ không phân trang.
- **get_object()**: trả về thể hiện của đối tượng được sử dụng cho detail.

40

40



Generic View cụ thể

- CreateAPIView
- ListAPIView
- RetrieveAPIView
- DestroyAPIView
- UpdateAPIView
- ListCreateAPIView
- RetrieveUpdateAPIView
- RetrieveDestroyAPIView
- RetrieveUpdateDestroyAPIView

41

41



Các lớp Mixins

- Các lớp mixins (`rest_framework.mixins`) cung cấp các action **hiện thực sẵn** các hành vi view cơ bản.
- Một số lớp mixins
 - ListModelMixin: hiện thực sẵn `list()`.
 - CreateModelMixin: hiện thực sẵn `create()`.
 - RetrieveModelMixin: hiện thực sẵn `retrieve()`.
 - UpdateModelMixin: hiện thực sẵn `update()` và `update_partial()`
 - DestroyModelMixin: hiện thực sẵn `destroy()`.

42

42

- Django REST framework cho phép **kết hợp logic của các view** liên quan trong một lớp duy nhất gọi là **ViewSet**.
- ViewSet **không cung cấp** các phương thức get(), post(), **thay vào đó** là các phương thức như list(), create(), retrieve(), destroy(), update().
- Chú ý: các lớp ViewSet **không hiện thực sẵn** các action.

- Các phương thức của ViewSets

```
class LessonViewSet(viewsets.ViewSet):
    def list(self, request):
        pass
    def create(self, request):
        pass
    def retrieve(self, request, pk=None):
        pass
    def update(self, request, pk=None):
        pass
    def partial_update(self, request, pk=None):
        pass
    def destroy(self, request, pk=None):
        pass
```

```

class LessonViewSet(viewsets.ViewSet):
    def list(self, request):
        lessons = Lesson.objects.filter(active=True)
        serializer = LessonSerializer(lessons, many=True)

        return Response(data=serializer.data)
    def retrieve(self, request, pk):
        try:
            lesson = Lesson.objects.get(pk=pk)
        except Lesson.DoesNotExist:
            return Http404()

        return Response(LessonSerializer(lesson).data)
    def create(self, request):
        d = request.data
        l = Lesson.objects.create(subject=d['subject'],
                                   content=d['content'],
                                   course_id=d['course id'])
        serializer = LessonSerializer(l)

        return Response(serializer.data,
                        status=status.HTTP_201_CREATED)

```

45



ViewSet

- Một số thuộc tính trong ViewSet
 - **basename**: tên URL được tạo.
 - **action**: tên các hành vi trong ViewSet như list, retrieve, create.
 - **name**: tên của ViewSet.
 - **description**: mô tả View của ViewSet.
 - **detail**: chỉ định action hiện tại có được cấu hình cho list và detail không.
 - **suffix**: phần đuôi của ViewSet type.

46

46

- Kiểm tra quyền trong LessonViewSet như sau:

```
class LessonViewSet(viewsets.ViewSet):  
    def get_permissions(self):  
        if self.action in ['list', 'retrieve']:  
            return [IsAuthenticated()]  
  
        return [IsAdminUser()]  
  
    ...
```

47

47

- Ta **đăng ký ViewSet** cho lớp **router**, nó sẽ tự động tạo các urlconfs.

```
from django.urls import path, include  
from . import views  
from rest framework import routers  
  
router = routers.DefaultRouter()  
router.register('courses', views.CourseViewSet)  
router.register('lessons', views.LessonViewSet,  
               basename='lesson')  
  
urlpatterns = [  
    path('', include(router.urls))  
]
```

48

48



Routers

- Hai đối số bắt buộc của register()
 - **prefix**: phần tên trước trên URL được sử dụng cho tất cả router, dạng {prefix}/
 - **viewset**: lớp ViewSet.
- Đối số tùy chọn
 - **basename**: tên URL được tạo ra, mặc định dựa trên thuộc tính queryset trong ViewSet.

49

49



Routers

- Với khai báo ViewSet này, ta có các API sau:
 - **/lessons/** – GET (list): lấy danh sách bài học, có tên là **lesson-list**.
 - **/lessons/** – POST (create): thêm bài học, có tên là **lesson-add**.
 - **/lessons/{pk}/** – GET (retrieve): xem chi tiết thông tin một bài học, có tên là **lesson-detail**.

50

50

Routers

Lesson List - Django REST framework

Api Root / Lesson List

Lesson List

GET /lessons/

HTTP 200 OK
Allow: GET, POST, HEAD, OPTIONS
Content-Type: application/json
Vary: Accept

```

{
  "id": 1,
  "subject": "Giới thiệu",
  "content": "Giới thiệu các công nghệ lập trình hiện đại",
  "course_id": 1,
  "created_date": "2021-05-30T01:59:15.931582Z",
  "updated_date": "2021-05-30T07:41:17.871208Z",
  "tags": []
}

```

Thực thi list

Media type: application/json

Content:

POST

Thực thi create

Lesson Instance - Django REST framework

Api Root / Lesson List / Lesson Instance

Lesson Instance

GET /lessons/1/

HTTP 200 OK
Allow: GET, HEAD, OPTIONS
Content-Type: application/json
Vary: Accept

```

{
  "id": 1,
  "subject": "Giới thiệu",
  "content": "Giới thiệu các công nghệ lập trình hiện đại",
  "course_id": 1,
  "created_date": "2021-05-30T01:59:15.931582Z",
  "updated_date": "2021-05-30T07:41:17.871208Z",
  "tags": []
}

```

Thực thi retrieve

51

Routers

Media type: application/json

Content:

```

{
  "subject": "React Native",
  "content": "Phát triển Hybrid App với React Native",
  "course_id": 1
}

```

POST

Api Root / Lesson List

Lesson List

POST /lessons/

HTTP 201 Created
Allow: GET, POST, HEAD, OPTIONS
Content-Type: application/json
Vary: Accept

```

{
  "id": 6,
  "subject": "React Native",
  "content": "Phát triển Hybrid App với React Native",
  "course_id": 1,
  "created_date": "2021-06-07T03:24:52.157221Z",
  "updated_date": "2021-06-07T03:24:52.157263Z",
  "tags": []
}

```

Kết quả response →

52



ModelViewSet

- ModelViewSet cung cấp **một số xử lý mặc định** cho ViewSet.
- Ví dụ tạo ModelViewSet cho Lesson.

```
class LessonViewSet(viewsets.ModelViewSet):  
    serializer_class = LessonSerializer  
    queryset = Lesson.objects.filter(active=True)
```

- Bổ sung các urls của ViewSet này vào route của REST framework.

53

53



ModelViewSet

- Các API sau được tạo ra tự động
 - /lessons/ - GET
 - /lessons/ - POST
 - /lessons/{id}/ - GET
 - /lessons/{id}/ - PUT
 - /lessons/{id}/ - PATCH
 - /lessons/{id}/ - DELETE

54

54

ModelViewSet

- **Thêm action** vào ViewSet, bổ sung thêm API ẩn bài học (cập nhật active thành False)

```
from rest_framework.decorators import action

class LessonViewSet(viewsets.ModelViewSet):
    @action(methods=['post'], detail=True)
    def hide_lesson(self, request, pk=None):
        ...
```

- Mặc định API được tạo (ta có thể đổi thông tin URL bằng thuộc tính **url_path** và **url_name**)
 - /lessons/{pk}/hide_lesson/ - POST

55

55

ModelViewSet

```
class LessonViewSet(viewsets.ModelViewSet):
    ...
    @action(methods=['post'], detail=True,
            name='Hide this lesson',
            url_path='hide-lesson',
            url_name='hide-lesson')
    def hide_lesson(self, request, pk=None):
        try:
            lesson = Lesson.objects.get(pk=pk)
            lesson.active = False
            lesson.save()
        except Course.DoesNotExist:
            return Response(status=status.HTTP_400_BAD_REQUEST)

        serializer = LessonSerializer(lesson)
        return Response(serializer.data,
                        status=status.HTTP_200_OK)
```

56

56

ModelViewSet

- Truy cập vào một bài học cụ thể

Lesson Instance – Django REST framework

127.0.0.1:8000/lessons/1/

Api Root / Lesson List / Lesson Instance

Lesson Instance

Extra Actions ▾ DELETE OPTIONS GET ▾

Hide this lesson

GET /lessons/1/

HTTP 200 OK

Allow: GET, PUT, PATCH, DELETE, HEAD, OPTIONS

Content-Type: application/json

Vary: Accept

```
{
  "id": 1,
  "subject": "Giới thiệu",
  "content": "Giới thiệu các công nghệ lập trình hiện đại",
  "course_id": 1,
  "created_date": "2021-05-30T01:59:15.931582Z",
  "updated_date": "2021-05-30T07:41:17.071208Z",
  "tags": [
    {
      "id": 1,
      "name": "nhập môn"
    }
  ]
}
```

57

57

ModelViewSet

Hide this lesson – Django REST framework

127.0.0.1:8000/lessons/1/hide-lesson/

Api Root / Lesson List / Lesson Instance / Hide this lesson

Hide this lesson

Extra Actions ▾ OPTIONS

GET /lessons/1/hide-lesson/

HTTP 405 Method Not Allowed

Allow: POST, OPTIONS

Content-Type: application/json

Vary: Accept

```
{
  "detail": "Method \"GET\" not allowed."
}
```

Raw data HTML form

Media type: application/json

Content:

```
{
  "subject": "",
  "content": "",
  "tags": []
}
```

API không yêu cầu thông tin gì trong body, nên chỉ cần click POST để thực thi ẩn bài viết

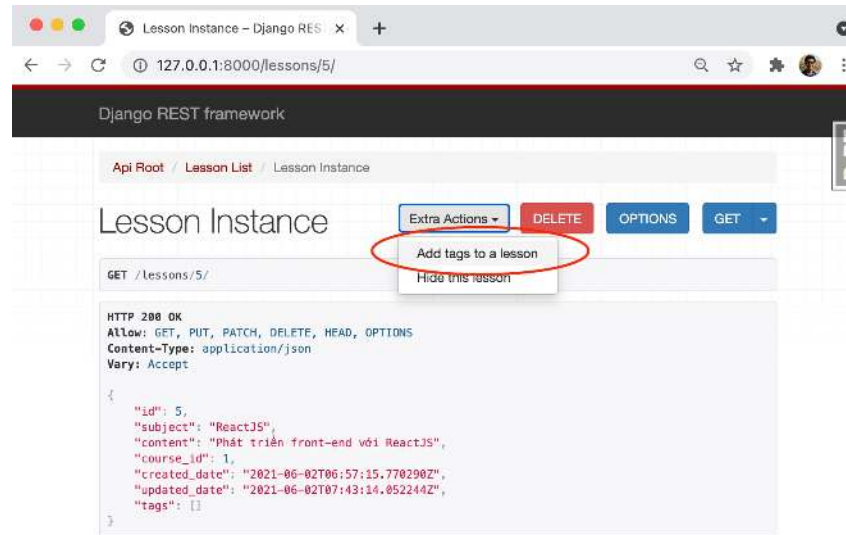
POST

58

58

ModelViewSet

- Thêm action cho phép gắn tag vào cho lesson.



59

```
class LessonViewSet(viewsets.ModelViewSet):
    @action(methods=['post'], detail=True,
            name="Add tags to a lesson",
            url_path='add-tags-to-lesson',
            url_name='add-tags')
    def add_tags_to_lesson(self, request, pk):
        try:
            lesson = Lesson.objects.get(pk=pk)

            tags = request.data.get('tags')
            for tag in tags.split(','):
                t, _ = Tag.objects.get_or_create(name=tag.strip())
                lesson.tags.add(t)
            lesson.save()
        except Lesson.DoesNotExist | KeyError:
            return Response(status=status.HTTP_404_NOT_FOUND)

        serializer = LessonSerializer(lesson)
        return Response(serializer.data,
                        status=status.HTTP_200_OK)
```

60

ModelViewSet

The screenshot shows the Django REST framework API browser interface. The browser address bar displays the URL `127.0.0.1:8000/lessons/5/add-tags-to-lesson/`. The page title is "Add tags to a lesson". The breadcrumb navigation shows "Api Root / Lesson List / Lesson Instance / Add tags to a lesson". The HTTP status is "HTTP 405 Method Not Allowed". The allowed methods are "POST, OPTIONS". The content type is "application/json". The response body is a JSON object: `{ "detail": "Method 'GET' not allowed." }`. The "Media type" is set to "application/json". The "Content" field contains `{"tags": "reactjs, redux"}`, which is circled in red. The "POST" button is visible at the bottom right.

61

61

ModelViewSet

The screenshot shows the Django REST framework API browser interface. The browser address bar displays the URL `127.0.0.1:8000/lessons/5/add-tags-to-lesson/`. The page title is "Add tags to a lesson". The breadcrumb navigation shows "Api Root / Lesson List / Lesson Instance / Add tags to a lesson". The HTTP status is "HTTP 200 OK". The allowed methods are "POST, OPTIONS". The content type is "application/json". The response body is a JSON object: `{ "id": 5, "subject": "ReactJS", "content": "Phát triển front-end với ReactJS", "course_id": 1, "created_date": "2021-06-02T06:57:15.778290Z", "updated_date": "2021-06-07T06:58:42.556182Z", "tags": [ { "id": 5, "name": "reactjs" }, { "id": 6, "name": "redux" } ] }`. The response body is circled in red and labeled "Response Data".

62

62



Parsers

- REST cung nhiều lớp Parser cho phép xử lý các request với nhiều media type khác nhau.
- Khi nhận request.data, REST dựa trên Content-Type của request để xác định sử dụng parser nào để parse nội dung request.
- Ta có thể thiết lập parser mặc định toàn cục cho ứng dụng trong settings giống như sau:

```
REST_FRAMEWORK = {  
    'DEFAULT_PARSER_CLASSES': [  
        'rest_framework.parsers.JSONParser',  
    ]  
}
```

63

63



Parsers

- Ví dụ sử dụng FileUploadParser cho view đăng ký người dùng vì cần upload ảnh đại diện.

```
from rest_framework.parsers import MultiPartParser  
  
class UserViewSet(viewsets.ViewSet,  
                  generics.CreateAPIView):  
    serializer_class = UserSerializer  
    queryset = User.objects.filter(is_active=True)  
    parser_classes = [MultiPartParser, ]
```

64

64



Authentication

- Authentication là cơ chế kết hợp các request nhận được với tập các thông tin chứng thực.
- Các chính sách permission và throttling sử dụng các thông tin chứng thực để quyết định request có quyền truy cập không.
- Authentication luôn **chạy khi bắt đầu view** và **trước khi kiểm tra permission** và throttling.

65

65



Authentication

- **request.user** là thể hiện của lớp User thuộc gói contrib.auth.
- **request.auth** chứa thông tin chứng thực khác, chẳng hạn lưu token.

66

66



Thực hiện Authentication

- Các loại chứng thực (authentication schemes) được định nghĩa danh sách lớp python.
- REST framework sẽ **chứng thực mỗi lớp trong danh sách và thiết lập giá trị request.user và request.auth** cho lớp đầu tiên được chứng thực thành công.
- Nếu không có lớp nào chứng thực thành công
 - request.user là thể hiện của `django.contrib.auth.models.AnonymousUser`
 - `Request.auth = None`

67

67



Thực hiện Authentication

- Cấu hình **DEFAULT_AUTHENTICATION_CLASSES** trong biến `REST_FRAMEWORK` trong `settings.py` để **chỉ định các lớp mặc định** dùng chứng thực.

```
REST_FRAMEWORK = {  
    'DEFAULT_AUTHENTICATION_CLASSES': [  
        'rest_framework.authentication.BasicAuthentication',  
        'rest_framework.authentication.TokenAuthentication'  
    ]  
}
```

68

68



Thực hiện Authentication

- Ngoài ra, ta có thể thiết lập các lớp chứng thực trong các lớp view thông qua thuộc tính **authentication_classes**

```
class UserView(APIView):  
    authentication_classes = [BasicAuthentication,  
                             TokenAuthentication]  
    permission_classes = [IsAdminUser]  
  
    def get(self, request):  
        pass
```

69

69



Thực hiện Authentication

- Nếu một request không được chứng thực thì có 2 error code sau đây phù hợp
 - **HTTP 401 – Unauthorized**: luôn có header WWW-Authenticate hướng dẫn client làm thế nào chứng thực.
 - **HTTP 403 – Permission Denied**
- Chú ý khi request được chứng thực thành công, nhưng **không có quyền** truy cập tài nguyên nào đó thì **HTTP 403** luôn được sử dụng.

70

70



Các lớp Authentication

- **BasicAuthentication**: chứng thực người dùng thông qua **username và password**. Nếu chứng thực thành công thì
 - request.user là thể hiện của User của Django.
 - request.auth = None
 - Nếu request không được chứng thực thì **HTTP 401 Unauthorized** sẽ được trả về.

71

71



Các lớp Authentication

- **TokenAuthentication**: chứng thực người dùng thông qua **token**, phù hợp cho ứng dụng theo **kiến trúc client-server**.
- Để sử dụng lớp chứng thực này, ta cần cấu hình thêm cho biến **INSTALLED_APP** như sau:

```
INSTALLED_APPS = [  
    ...  
    'rest_framework.authtoken'  
]
```

- Thực thi lệnh migrate.

72

72



Các lớp Authentication

- Tạo token cho user

```
from rest_framework.authtoken.models import Token  
  
token = Token.objects.create(user=...)   
print(token.key)
```

- Phía client chứng thực thiết lập Authorization trong **headers** như sau:

```
Authorization: Token <token-value>
```

- Khi đó request.auth là thể hiện của rest_framework.authtoken.models.Token

73

73



Các lớp Authentication

- **SessionAuthentication**: lớp chứng thực này sử dụng session của django. Lớp chứng thực này phù hợp cho client dùng ajax chạy trong cùng ngữ cảnh session của website.
- **RemoteUserAuthentication**: lớp chứng thực này cho phép uỷ quyền chứng thực cho web server, thiết lập cho biến môi trường REMOTE_USER.

74

74



OAuth2

- OAuth là một giao thức mở và an toàn để **chứng thực người dùng giữa các dịch vụ liên quan**.
- OAuth **ủy quyền** một dịch vụ được phép truy cập những tài nguyên từ một dịch vụ khác đại diện cho người dùng, mà **không cần** cung cấp thông tin như username, password.
- OAuth2 là phiên bản mới nhất của giao thức OAuth.

75

75



OAuth2

- **Client**: ứng dụng **thực hiện hành vi** đại diện người dùng truy cập các tài nguyên.
- **Resource Owner/User**: là **người dùng ủy quyền** cho ứng dụng **truy cập** vào các tài nguyên được bảo vệ bởi nhà cung cấp khác.
- **Authorization Server**: server chứng thực yêu cầu từ phía client.
- **Resource Server**: server chứa các tài nguyên được bảo vệ.

76

76

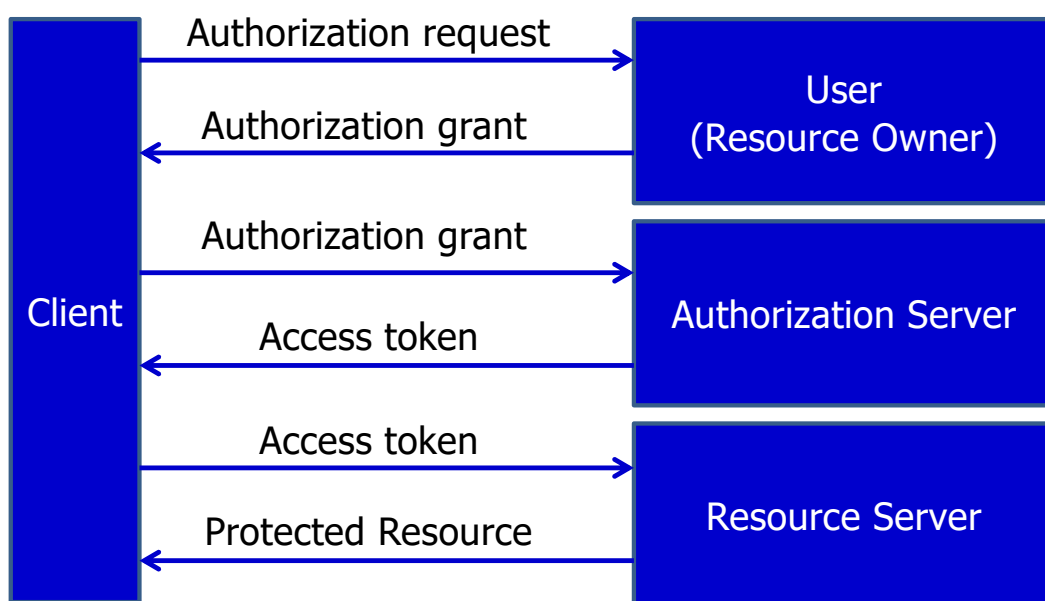
OAuth2

- Các bước thực hiện giao tiếp giữa client và server theo giao thức OAuth2:
 - Authorize: chứng thực yêu cầu quyền truy cập từ client.
 - Yêu cầu access token.
 - Truy cập các tài nguyên được bảo vệ.

77

77

OAuth2



78

78



OAuth2

- Trước khi sử dụng OAuth2 để chứng thực, người phát triển (developer) phải **đăng ký một application** với OAuth2 provider.
- Sau khi đăng ký xong sẽ được cung cấp thông tin: **client ID**, **client secret**, authorization server URL, access token URL, resource server URL.

79

79



OAuth2

- Client ID: thông tin công khai định danh ứng dụng, giống như username.
- Client secret: thông tin riêng tư gửi kèm lên server, giống như password.
- Authorization server URL: URL để client thực hiện các yêu cầu (request).
- Access token URL: URL để client yêu cầu access token.
- Resource server URL: URL để client truy cập các tài nguyên được bảo vệ.

80

80



Django OAuth Toolkits

- Django OAuth Toolkit cung cấp tất cả các endpoint, dữ liệu và logic cần thiết để thực hiện giao thức OAuth2 cho django project.
- Cài đặt

```
pip install django-oauth-toolkit
```

81

81



Django OAuth Toolkits

- Cập nhật biến INSTALLED_APP trong settings.py

```
INSTALLED_APPS = (  
    ...  
    'oauth2_provider',  
)
```

- Bổ sung thông tin cấu hình cho biến REST_FRAMEWORK trong settings.py

```
REST_FRAMEWORK = {  
    ...  
    'DEFAULT_AUTHENTICATION_CLASSES': (  
        'oauth2_provider.contrib.rest_framework.OAuth2Authentication',  
    )  
}
```

82

82

Django OAuth Toolkits

- Cập nhật urls cho URLConfig của project

```
from django.urls import include, path

urlpatterns = [
    ...
    path('o/', include('oauth2_provider.urls',
                      namespace='oauth2_provider')),
]
```

- Thực thi migrate

```
python manage.py migrate oauth2_provider
```

83

83

Django OAuth Toolkits

- Truy cập: <http://127.0.0.1:8000/o/applications/>

The screenshot shows a web browser window with the URL <http://127.0.0.1:8000/o/applications/register/>. The page displays a form titled "Register a new application". The form fields are: Name (CourseApp), Client id (kchca8lucgNiA9e3yFISJhnOHfx3QCO1gFAI1t), Client secret (VolMCkxrbwMebFmBoCGEuTxHmd9l28t1t7EC8uoGo1hHWb6dZWNLU4B6), Client type (Confidential), Authorization grant type (Resource owner password based), Redirect uri (empty), and Algorithm (No OIDC support). There are "Go Back" and "Save" buttons at the bottom.

The screenshot shows a web browser window with the URL localhost:8000/o/applications/1/. The page displays the details for the "CourseApp" application. The fields shown are: Client id (kchca8lucgNiA9e3yFISJhnOHfx3QCO1gFAI1t), Client secret (VolMCkxrbwMebFmBoCGEuTxHmd9l28t1t7EC8uoGo1hHWb6dZWNLU4B6), Client type (confidential), Authorization Grant Type (password), and Redirect Uri (empty). There are "Go Back", "Edit", and "Delete" buttons at the bottom.

84

Django OAuth Toolkits

- Yêu cầu lấy access token
 - Url: **/o/token/**
 - Method: POST
 - Body data

```
{  
  "grant_type": "password",  
  "username": "<username>",  
  "password": "<password>",  
  "client_id": "<client-id>",  
  "client_secret": "<client-secret>"  
}
```

85

85

Django OAuth Toolkits

The screenshot shows the Postman interface for a POST request to `http://localhost:8000/o/token/`. The request body is set to form-data with the following fields:

KEY	VALUE
grant_type	password
username	admin
password	thanh@123
client_id	lcchca8luucgNIAJ9e3yFISJhnOHfx3QCO1gFAIt
client_secret	VcIMCkarbwMelbFmIbCoCGEuTxHmd9I28t17EC8ucQ

The response is shown in JSON format:

```
{  
  "access_token": "JAdD49pzqs73dwHzq3EtaTgaI74qt5",  
  "expires_in": 36000,  
  "token_type": "Bearer",  
  "scope": "read write",  
  "refresh_token": "KkBkye6WpxCoh89hTzTjC0PmMVLbJs"  
}
```

86



Django OAuth Toolkits

- Sau khi có **access token**, ta có thể sử dụng để yêu cầu truy cập một số **tài nguyên yêu cầu chứng thực** mà không cần gửi username, password.
- Trong HTTP header của mỗi request phải gửi kèm Authorization, giá trị của nó có thể là **Token** hoặc **Bearer** phụ thuộc OAuth2 provider.
- Cú pháp
 - **Authorization: Bearer <access-token>**

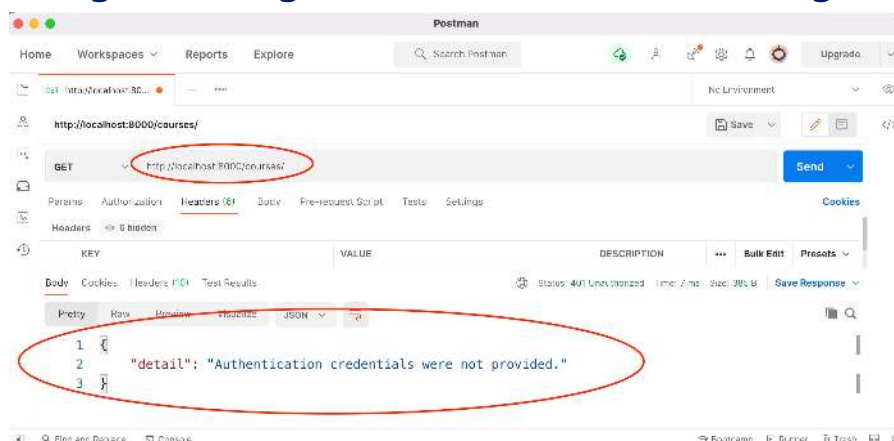
87

87



Django OAuth Toolkits

- Phần trước ta đã cấu hình cho API /courses/ chỉ được truy cập khi đăng nhập, giờ truy cập vào không có thông tin access token để chứng thực.



88

88

Django OAuth Toolkits

The screenshot shows the Postman interface for a GET request to `http://localhost:8000/courses/`. The **Headers** tab is selected, showing a table with one header:

KEY	VALUE	DESCRIPTION
Authorization	Bearer JAdD48pzs73dwhzq3EtaTgal74qt5	

Red circles and text highlight the header: "Thông tin access token đặt trong headers của request". The response body is a JSON object:

```
{
  "count": 8,
  "next": "http://localhost:8000/courses/?page=2",
  "previous": null,
  "results": [
    {
      "id": 8,
      "subject": "Cơ sở lập trình",
      "created_date": "2021-06-01T03:14:17.053760Z",
      "category_id": 1
    }
  ]
}
```

89

CORS

- CORS (**Cross-origin resource sharing**) là cơ chế cho phép **giới hạn các tài nguyên** trên trang web được yêu cầu truy cập từ **domain khác**.
- CORS cho phép trình duyệt và server xác định được request cross-origin có an toàn không.

90

90

CORS

- Cài cors middleware

```
pip install django-cors-headers
```

- Bổ sung biến INSTALLED_APP trong settings.py

```
INSTALLED_APP = [  
    ...,  
    'corsheaders'  
]
```

91

91

CORS

- Cập nhật biến MIDDLEWARE của settings.py giá trị corsheaders.middleware.CorsMiddleware. Giá trị này nên đặt trước các middleware có thể tạo ra response.

```
MIDDLEWARE = [  
    'corsheaders.middleware.CorsMiddleware',  
    ...  
]
```

- Thiết lập giá trị biến sau trong settings.py

```
CORS_ALLOW_ALL_ORIGINS = True
```

92

92



JSON Web Token

- JSON Web Token (JWT) là tiêu chuẩn chứng thực mới dựa trên token, nhưng không giống với TokenAuthentication có sẵn, JWT **không sử dụng CSDL** để kiểm tra token.
- Ở đây minh họa sử dụng JWT thông qua thư viện Simple JWT.

93

93



Simple JWT

- Cài đặt

```
pip install djangorestframework-simplejwt
```

- Chỉ định lớp chứng thực trong settings.py

```
REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': (
        ...
        'rest_framework_simplejwt.authentication.JWTAuthentication',
    )
}
```

94

94



Simple JWT

- ```
from rest_framework.simplejwt.views import (
 TokenObtainPairView,
 TokenRefreshView
)

urlpatterns = [
 ...
 path('api/token/', TokenObtainPairView.as_view(),
 name='token_obtain_pair'),
 path('api/token/refresh/', TokenRefreshView.as_view(),
 name='token_refresh'),
]
```

95



# Simple JWT

The screenshot shows a Postman interface for a Django REST API. The request is a POST to `http://127.0.0.1:8000/api/token/`. The body data is in the following table:

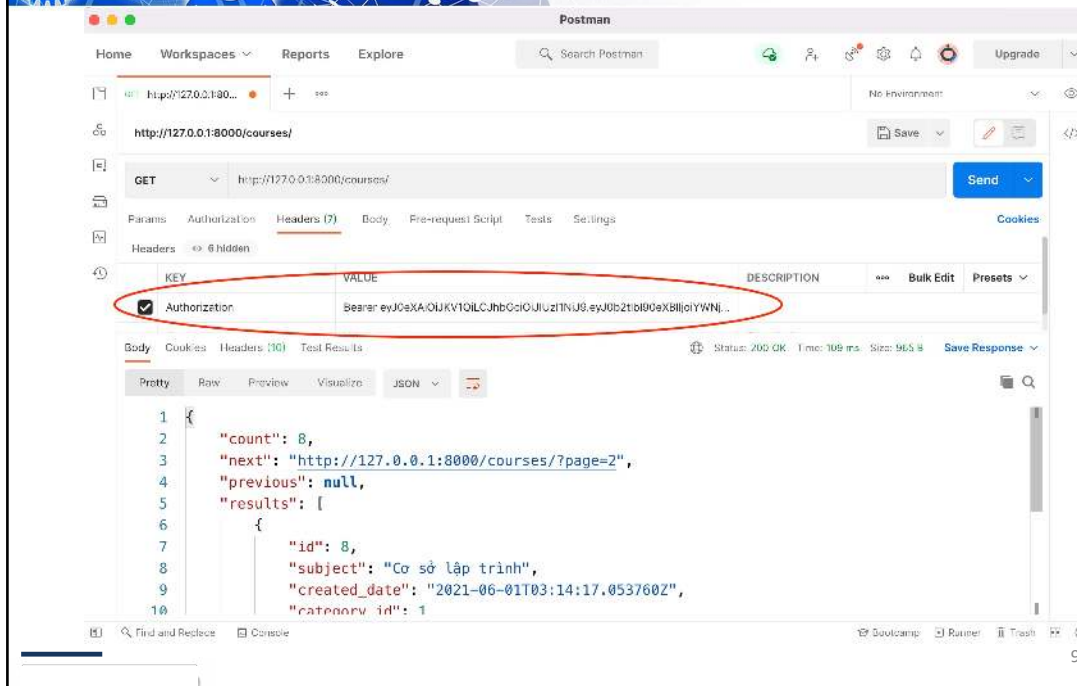
| KEY      | VALUE      | DESCRIPTION |
|----------|------------|-------------|
| username | admin      |             |
| password | thanhh@123 |             |

The response data is a JSON object:

```
{
 "refresh": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ0b2t1b190eXBlijoicmVmczZlciImV4cCI6MTYyMjg1MDg2NCwianRpIjoimDVkZDA0NW5ZWm4NG05ZWFhInQwZjU4NTYyNWYyZDQlCl1cY2VYX2lkIjoxfQ.1rKev7RphNhh2aB1XzUSg1b6jDYsJhD1b-bGLhJ1DpLo",
 "access": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ0b2t1b190eXBlijoicmVmczZlciImV4cCI6MTYyMjg1MDg2NCwianRpIjoimDVkZDA0NW5ZWm4NG05ZWFhInQwZjU4NTYyNWYyZDQlCl1cY2VYX2lkIjoxfQ.1rKev7RphNhh2aB1XzUSg1b6jDYsJhD1b-bGLhJ1DpLo"
}
```

96

## Simple JWT



97

## Tích hợp Swagger

- Cài đặt drf-yasg (Yet Another Swagger Generator)

```
pip install drf-yasg
```

- Cập nhật biến INSTALLED\_APP

```
INSTALLED_APPS = [
 ...
 'django.contrib.staticfiles',
 'drf_yasg',
]
```

98

98



## Tích hợp Swagger

- Cập nhật urls cho **URLConfs** của project

```
from rest_framework import permissions
from drf_yasg.views import get_schema_view
from drf_yasg import openapi

schema_view = get_schema_view(
 openapi.Info(
 title="Course API",
 default_version='v1',
 description="APIs for CourseApp",
 contact=openapi.Contact(email="thanh.dh@ou.edu.vn"),
 license=openapi.License(name="Dương Hữu Thành@2021"),
),
 public=True,
 permission_classes=(permissions.AllowAny,),
)
```

99



## Tích hợp Swagger

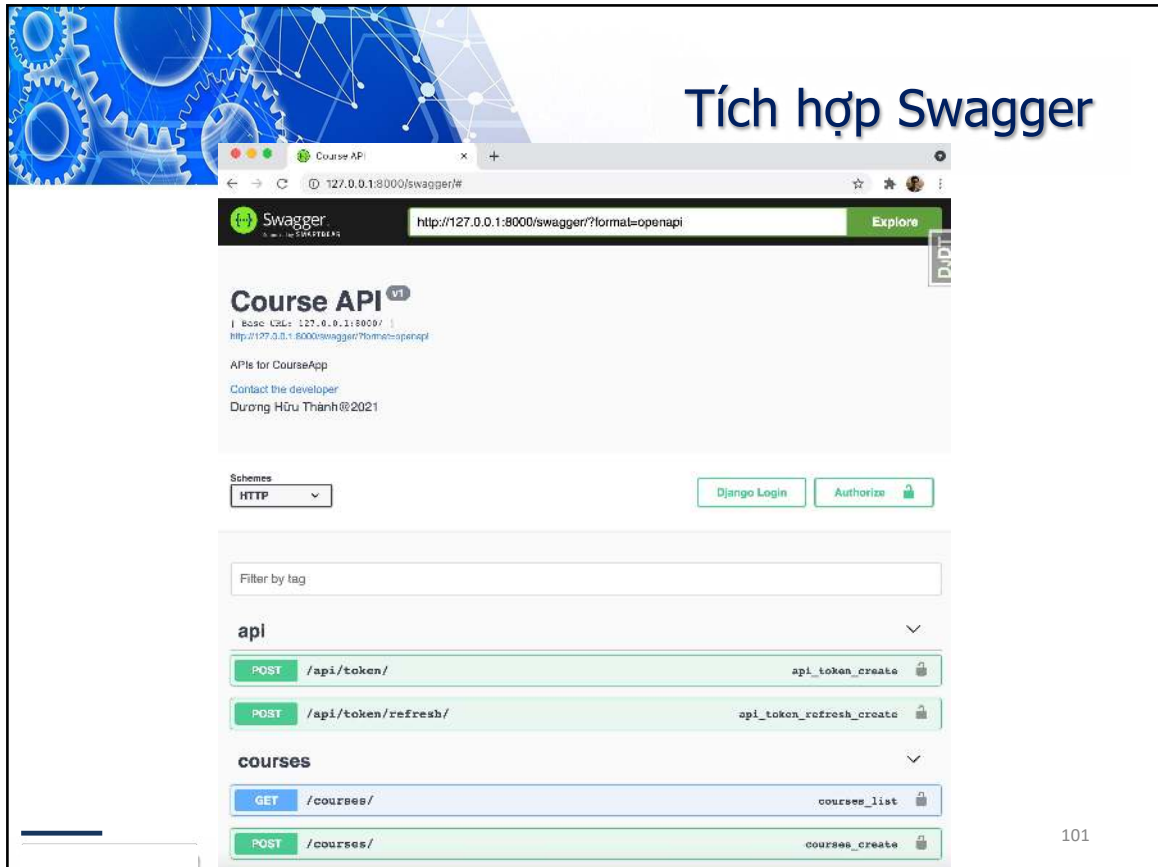
- Cập nhật urls cho URLConf của project

```
...
urlpatterns = [
 ...
 re_path(r'^swagger(?:P<format>\.json|\.yaml)$',
 schema_view.without_ui(cache_timeout=0),
 name='schema-json'),
 re_path(r'^swagger/$',
 schema_view.with_ui('swagger', cache_timeout=0),
 name='schema-swagger-ui'),
 re_path(r'^redoc/$',
 schema_view.with_ui('redoc', cache_timeout=0),
 name='schema-redoc')
]
```

100

100

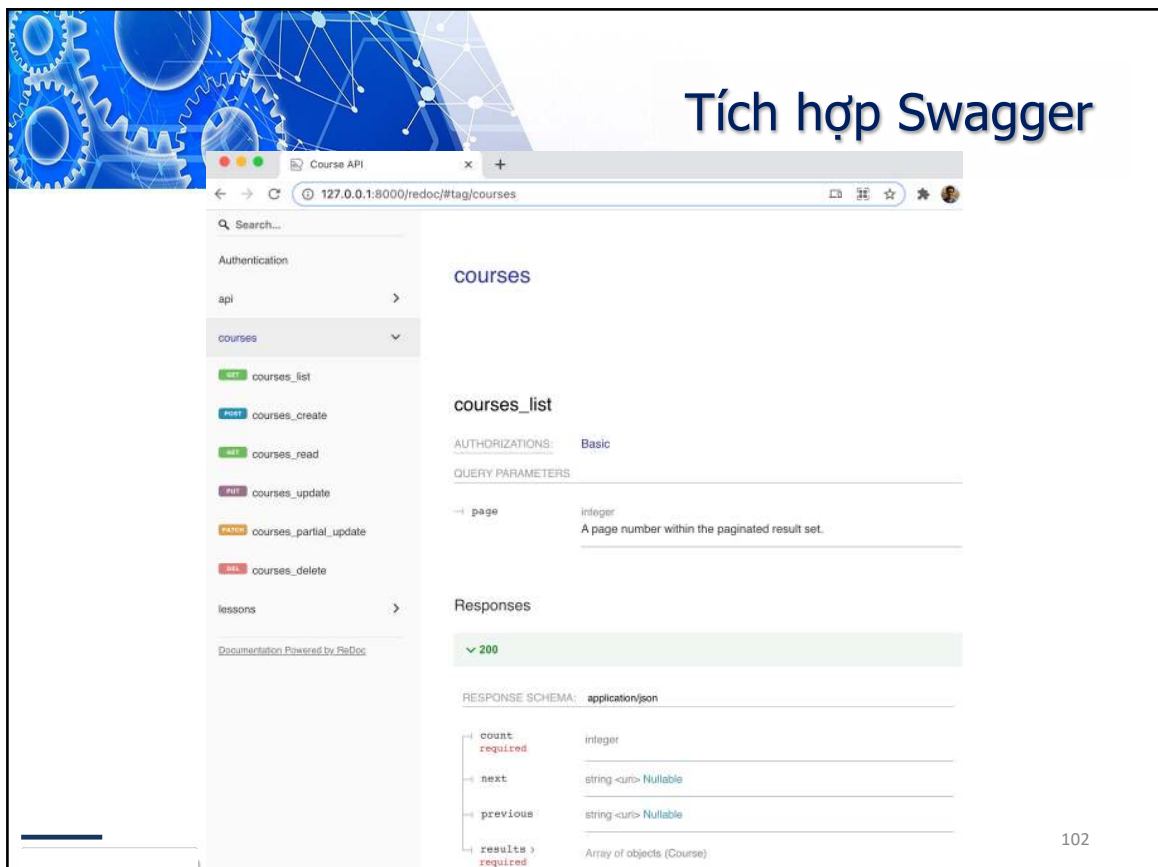
# Tích hợp Swagger



101

101

# Tích hợp Swagger



102

102

## Tích hợp Swagger

- Ta có thể thiết lập ẩn một api để không đưa vào swagger bằng cách thiết lập thuộc tính `swagger_schema = None`

```
class UserList(APIView):
 swagger_schema = None
 ...
```

103

103

## Tích hợp Swagger

```
from drf_yasg.utils import swagger_auto_schema
from drf_yasg import openapi

class LessonViewSet(viewsets.ModelViewSet):
 @swagger_auto_schema(
 operation_description='Add tags to a lesson',
 responses={
 status.HTTP_200_OK: LessonSerializer()
 }
)
 @action(methods=['post'], detail=True,
 name="Add tags to a lesson",
 url_path='add-tags-to-lesson',
 url_name='add-tags')
 def add_tags_to_lesson(self, request, pk):
 ...
```

104

104





## Tích hợp Debug Toolbar

- Cài đặt

```
pip install django-debug-toolbar
```

- Cập nhật biến INSTALLED\_APP và STATIC\_URL

```
INSTALLED_APPS = [
 # ...
 'django.contrib.staticfiles',
 # ...
 'debug_toolbar',
]
STATIC_URL = '/static/'
```

105

105



## Tích hợp Debug Toolbar

- Debug Toolbar chủ yếu được thực hiện trong middleware. Trong settings.py bật middleware

```
MIDDLEWARE = [
 ...
 'debug_toolbar.middleware.DebugToolbarMiddleware',
]
```

- Debug Toolbar chỉ được hiển thị trên các địa chỉ IP có trong danh sách INTERNAL\_IPS của tập tin cấu hình.

```
INTERNAL_IPS = [
 '127.0.0.1'
]
```

106

106



## Tích hợp Debug Toolbar

- Cập nhật urls cho URLConfig của project

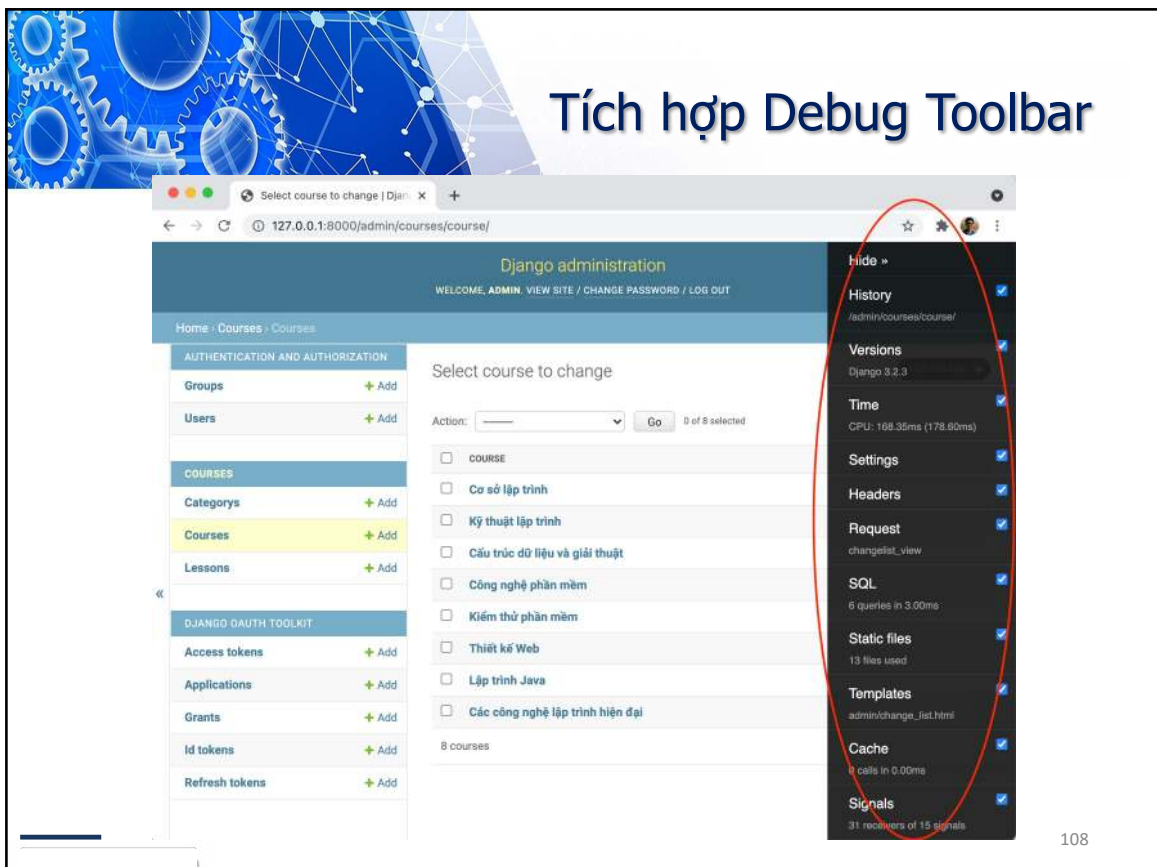
```
import debug_toolbar
from django.conf import settings
from django.urls import include, path

urlpatterns = [
 ...
 path('__debug__/', include(debug_toolbar.urls))
]
```

107

107

## Tích hợp Debug Toolbar



108

108

# Tích hợp Debug Toolbar

The screenshot shows a web browser window with the URL `127.0.0.1:8000/admin/courses/course/`. The page displays the Django Debug Toolbar, which is integrated into the admin interface. The toolbar shows the following information:

- SQL queries from 1 connection**: A table of SQL queries with columns for QUERY, TIMELINE, TIME (MS), and ACTION.
- History**: A list of recent requests, including `/admin/courses/course/`.
- Versions**: A list of installed Django versions, including `Django 3.2.3`.
- Time**: A summary of the current request's performance, including `CPU: 168.35ms (178.60ms)`.
- Settings**: A list of Django settings, including `admin/change_list.html`.
- Headers**: A list of request headers, including `change_list_view`.
- Request**: A summary of the current request, including `change_list_view`.
- SQL**: A summary of the current request's SQL queries, including `6 queries in 3.00ms`.
- Static files**: A summary of the current request's static files, including `13 files used`.
- Templates**: A summary of the current request's templates, including `admin/change_list.html`.
- Cache**: A summary of the current request's cache, including `0 calls in 0.00ms`.
- Signals**: A summary of the current request's signals, including `31 receivers of 15 signals`.